



Politechnika Łódzka

Instytut Automatyki

Zakład sterowania robotów

TWORZENIE SYSTEMÓW ROBOTYCZNYCH

Temat 1

Wprowadzenie do ROS2
Tworzenie nowego workspace'a i pakietu



10 marca 2025

Spis treści

1	Wprowadzenie do ROS2	2
1.1	Dlaczego warto poznać ROS2?	2
1.2	Przykłady zastosowań ROS2	2
2	Procedura tworzenia nowego workspace'a	3
2.1	Tworzenie katalogów dla workspace'a.	3
2.2	Przejsście do workspace'a.	3
2.3	Inicjalizacja workspace'a.	3
2.4	Source workspace'a.	4
2.5	Automatyczne dodanie workspace'a do ścieżki środowiskowej (opcjonalne)	4
2.6	Plik .bashrc	4
3	Procedura tworzenia nowego pakietu dla interfejsów komunikacyjnych.	5
3.1	Przejsście do katalogu src w workspace	5
3.2	Utworzenie nowego pakietu	5
3.3	Utworzenie podkatalogów	5
3.4	Utworzenie wiadomości dla ROS Topic (.msg)	6
3.5	Utworzenie wiadomości dla ROS Service (.srv)	6
3.6	Utworzenie wiadomości dla ROS Action (.action)	6
3.7	Dodanie interfejsu do CMakeLists.txt.	7
3.8	Modyfikacja package.xml	8
3.9	Budowanie pakietu	9
3.10	Weryfikacja działania	9
4	Procedura tworzenia nowego pakietu ROS2 dla wykonywalnych plików projektowych	10
4.1	Przejsście do katalogu src w workspace	10
4.2	Utworzenie nowego pakietu	11
4.3	Utworzenie podkatalogów	11
4.4	Modyfikacja package.xml w pakiecie example_tsr_project.	11
4.5	Modyfikacja setup.py w pakiecie example_tsr_project.	12
4.6	Budowanie paczki	13
4.7	Weryfikacja działania	13
5	Uruchomienie symulacji turtlesim	13
5.1	Najważniejsze tematy	13
5.2	Najważniejsze serwisy	14
6	Dokumentacja i inne linki	14

1 Wprowadzenie do ROS2

Wyobraź sobie, że masz możliwość stworzenia własnego robota – takiego, który samodzielnie porusza się po pokoju, unika przeszkód, rozpoznaje przedmioty i reaguje na polecenia. Brzmi jak coś z przyszłości? Dzięki ROS2 (Robot Operating System 2) – nowoczesnemu frameworkowi do programowania robotów – możesz to zrobić szybciej i łatwiej, niż myślisz!

ROS2 to zbiór narzędzi (m.in. do archiwizacji i analizy danych) i bibliotek (pakietów), które pomagają w programowaniu robotów. To coś w rodzaju „systemu operacyjnego dla robotów” – pozwala na komunikację między ich elementami, zarządzanie danymi z czujników, sterowanie ruchem i wiele więcej. Dzięki niemu nie musisz pisać wszystkiego od zera – możesz skorzystać z gotowych rozwiązań i skupić się na tym, co najważniejsze: tworzeniu funkcjonalnego i inteligentnego robota.

Możesz myśleć o ROS2 jak o zestawie klocków LEGO – każdy moduł odpowiada za inne zadanie (np. nawigację, sterowanie silnikami, analizę obrazu), a Twoim zadaniem jest połączenie ich w całość i dostosowanie do swoich potrzeb.

1.1 Dlaczego warto poznać ROS2?

- To przyszłość robotyki – ROS2 jest używany w robotach mobilnych, dronach, pojazdach autonomicznych i systemach przemysłowych. Jeśli chcesz pracować w tej branży, znajomość ROS2 to ogromna zaleta.
- Łatwość nauki i gotowe narzędzia – zamiast pisać skomplikowany kod od podstaw, możesz skorzystać z istniejących bibliotek i przykładów. ROS2 pozwala szybko testować pomysły, bez konieczności budowania fizycznego robota – można pracować w symulacji.
- Modularność – aplikacje w ROS2 składają się z małych, niezależnych programów (nazywanych węzłami), które komunikują się ze sobą. Dzięki temu łatwo rozszerzać projekt i dodawać nowe funkcje.
- Działa na różnych platformach – ROS2 obsługuje zarówno komputery PC, jak i Raspberry Pi oraz roboty mobilne. To oznacza, że możesz pisać kod na laptopie, a potem uruchomić go na prawdziwym robocie!
- Licencja open-source – ROS2 jest dostępny za darmo i rozwijany przez społeczność programistów, badaczy i firm z całego świata.

1.2 Przykłady zastosowań ROS2

- Boston Dynamics
https://www.youtube.com/watch?v=Kf63_hbmUWE&ab_channel=SaketPradhan
https://www.youtube.com/watch?v=yyGrHmtJWfU&ab_channel=LuisCruz
https://www.youtube.com/watch?v=mqDncPrTI2w&ab_channel=VentureX-FutureTech
- mini robot o kinematyce typu Ackermann
https://www.youtube.com/watch?v=22sJoi4Kg0&ab_channel=YahboomTechnology
- Symulacja - automatyczne dokowanie do stacji ładowania
https://www.youtube.com/watch?v=TY-FOH81yk4&ab_channel=AutomaticAddison

2 Procedura tworzenia nowego workspace'a

Przedstawiona procedura wykonywana jest w terminalu. Można go uruchomić `ctrl+alt+t` lub poprzez znalezienie w wyszukiwarce programów.

2.1 Tworzenie katalogów dla workspace'a.

W lokalizacji `~/tsr_workspaces` należy utworzyć nowy workspace o dowolnej nazwie. W tym celu dla przykładu utworzony zostanie katalog `example_tsr_ws` wraz z katalogiem `src`, w którym będą znajdowały się pakiety dla ROS2.

Dla innego workspace'a np. `new_workspace2` należy zmienić nazwę z `example_tsr_ws` na nazwę `new_workspace2`. W pozostałych poleceniach wykorzystujących workspace `example_tsr_ws` również należy wykonać taką zmianę.

```
1 mkdir -p ~/tsr_workspaces/example_tsr_ws/src
```

2.2 Przejście do workspace'a.

```
1 cd ~/tsr_workspaces/example_tsr_ws
```

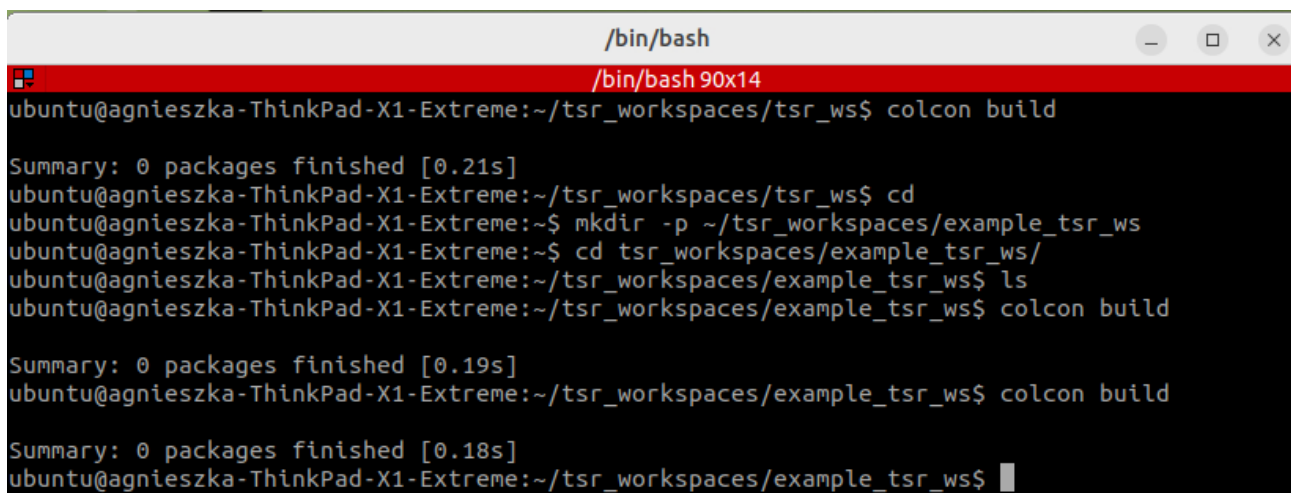
2.3 Inicjalizacja workspace'a.

Należy uruchomić pierwszą kompilację workspace'a (choć jeszcze nie zawiera pakietów). W wyniku, której utworzone zostają katalogi `build/`, `install/`, i `log/`. Kompilacja wykonywana jest zawsze w głównym katalogu workspace'a (Rys. 2.1), czyli w tym przypadku lokalizacja w terminalu wskazuje na katalog `/example_tsr_ws`.

W celu wykonania kompilacji należy użyć polecenia:

```
1 colcon build --symlink-install
```

`--symlink-install` nie jest wymagany, ale jego zastosowanie pozwala uniknąć ponownej kompilacji po wy-



```
/bin/bash
/bin/bash 90x14
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/tsr_ws$ colcon build
Summary: 0 packages finished [0.21s]
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/tsr_ws$ cd
ubuntu@agnieszka-ThinkPad-X1-Extreme:~$ mkdir -p ~/tsr_workspaces/example_tsr_ws
ubuntu@agnieszka-ThinkPad-X1-Extreme:~$ cd tsr_workspaces/example_tsr_ws/
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ls
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ colcon build
Summary: 0 packages finished [0.19s]
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ colcon build
Summary: 0 packages finished [0.18s]
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$
```

Rys. 2.1: Ścieżka w terminalu dla workspace'a

konaniu zmian w plikach Python i innych pikach niewymagających kompilacji.

2.4 Source workspace'a.

Aktywacja środowiska workspace'a, aby ROS2 widział jego zawartość. Umożliwia to korzystanie z utworzonych pakietów, które znajdują się w tym workspace (np. dostęp do utworzonych wiadomości, węzłów/nodów obsługujących różne funkcjonalności robota). Brak widoczności może być związany z brakiem source'a.

Aby to zrobić należy wpisać w terminalu (należy znajdować się w głównym katalogu workspace'a):

```
1 source install/setup.bash
```

Alternatywnie można wykorzystać pełną ścieżkę do setup.bash. W tym przypadku dla workspace'a /example_tsr_ws należy wpisać:

```
1 source ~/tsr_workspaces/example_tsr_ws/install/setup.bash
```

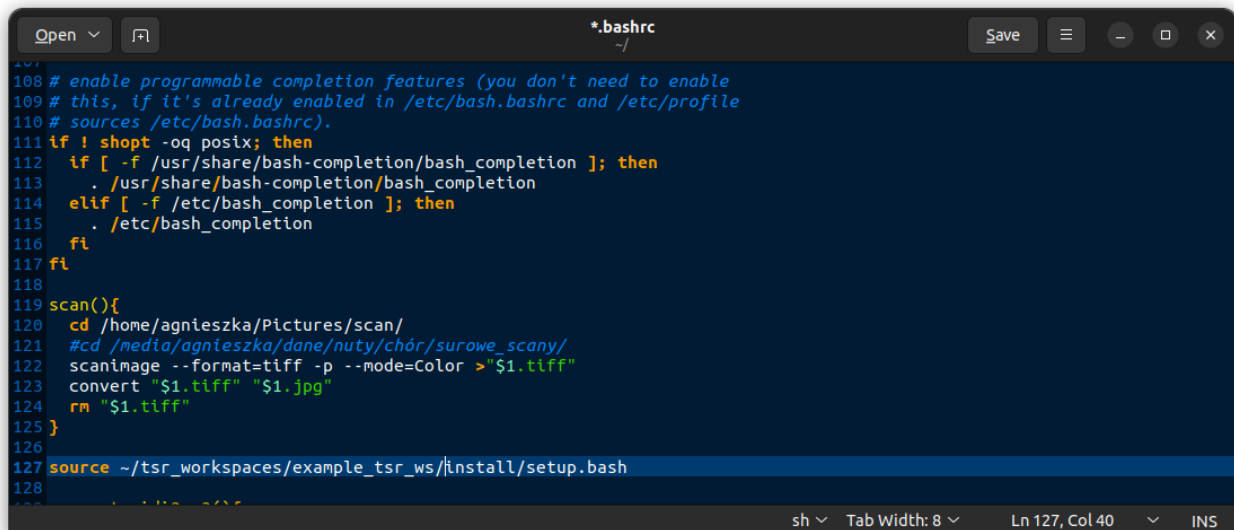
2.5 Automatyczne dodanie workspace'a do ścieżki środowiskowej (opcjonalne)

Dodanie automatycznego source spowoduje, że przy otwieraniu nowego terminala polecenie source wykona się automatycznie. Dla większej kontroli nad używanymi workspace'ami może nie być to konieczne. Jeśli chcemy dodać automatyczny source to polecenie należy wykonać tylko raz.

```
1 echo "source ~/tsr_workspaces/example_tsr_ws/install/setup.bash" >> ~/.bashrc
2 source ~/.bashrc
```

2.6 Plik .bashrc

Plik .bashrc (skrót od bash read command) to skrypt powłoki Bash, który wykonuje się automatycznie za każdym razem, gdy użytkownik otworzy nową sesję terminala. Poprzednie polecenie odpowiadało za dodanie linii source do bash (Rys. 2.2).



```

108 # enable programmable completion features (you don't need to enable
109 # this, if it's already enabled in /etc/bash.bashrc and /etc/profile
110 # sources /etc/bash.bashrc).
111 if ! shopt -oq posix; then
112   if [ -f /usr/share/bash-completion/bash_completion ]; then
113     . /usr/share/bash-completion/bash_completion
114   elif [ -f /etc/bash_completion ]; then
115     . /etc/bash_completion
116   fi
117 fi
118
119 scan(){
120   cd /home/agnieszka/Pictures/scan/
121   #cd /media/agnieszka/dane/nuty/chór/surowe_scan/
122   scanimage --format=tiff -p --mode=Color >"$1.tiff"
123   convert "$1.tiff" "$1.jpg"
124   rm "$1.tiff"
125 }
126
127 source ~/tsr_workspaces/example_tsr_ws/install/setup.bash
128

```

Rys. 2.2: Plik `.bashrc` z dodanym automatycznym `source` dla workspace'a `example_tsr_ws`

3 Procedura tworzenia nowego pakietu dla interfejsów komunikacyjnych.

Utworzona w tym kroku paczka z interfejsami komunikacyjnymi będzie służyła tylko i wyłącznie do dodawania nowych interfejsów dla ROS Topic, ROS Service i ROS Action wykorzystywanych w projekcie.

3.1 Przejście do katalogu `src` w workspace

Należy w terminalu przejść do workspace'a (`example_tsr_ws`) do katalogu:

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

3.2 Utworzenie nowego pakietu

Uruchomić polecenie odpowiedzialne za utworzenie nowego pakietu o nazwie `example_tsr_msgs`, w którym będą umieszczane nowe interfejsy komunikacyjne.

```
1 ros2 pkg create --build-type ament_cmake example_tsr_msgs
```

3.3 Utworzenie podkatalogów

Utworzenie w pakiecie `example_tsr_msgs` katalogów dla interfejsów komunikacyjnych dla ROS Topic (katalog `msg`), ROS Services (katalog `srv`) i ROS Action (katalog `action`). W tym celu należy przejść do lokalizacji głównego katalogu dla pakietu a następnie utworzyć katalog `msg`, `srv` i `action`.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs
2 mkdir msg srv action
```

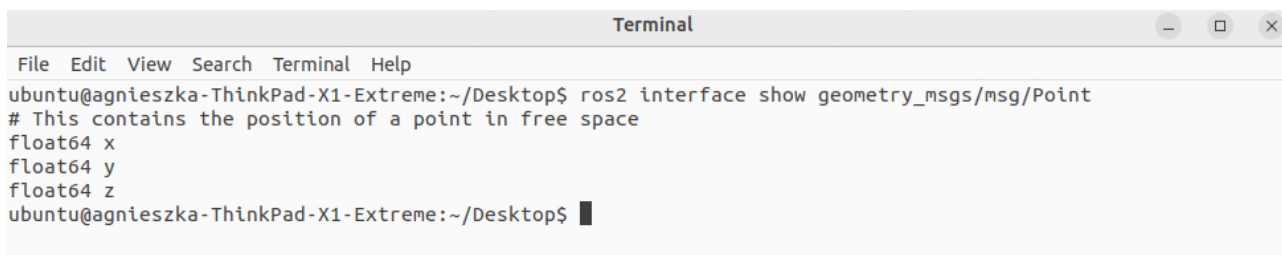
3.4 Utworzenie wiadomości dla ROS Topic (.msg)

W katalogu msg (`~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs/msg`) utworzyć plik tekstowy `ExampleMsgType.msg`, w którym zdefiniowany zostanie interfejs komunikacyjny dla wiadomości ROS Topic. Wszystkie pliki w tym katalogu będą miały rozszerzenie `.msg`. Do pliku należy wkleić przykład struktury wiadomości:

```
1 int32 id
2 string name
3 float32 value
4 bool is_valid
5 int32[] dynamic_integer_array
6 geometry_msgs/Point example_goal
```

Z lewej strony znajduje się typ pola wiadomości a z prawej nazwa pola, która jest wykorzystywana przy uzupełnianiu lub odczycie wartości z pola w wiadomości. W tym pliku znajduje się kilka przykładowych typów pól wiadomości. Pola zawierające listę elementów oznaczone są []. W tym przykładzie lista z wartościami typu `int_32` przypisana jest do pola `dynamic_integer_array`.

Oprócz podstawowych typów można używać bardziej zaawansowanych struktur pochodzących z innych pakietów. W tej wiadomości `example_goal` odnosi się do struktury wiadomości zdefiniowanej w pakiecie `geometry_msgs`.



Rys. 3.1: Struktura wiadomości `geometry_msgs/Point`

3.5 Utworzenie wiadomości dla ROS Service (.srv)

Utworzenie w katalogu `srv` podobnie jak dla wiadomości należy utworzyć plik tekstowy `ExampleSrvType.srv`.

Do pliku należy wkleić przykładowy interfejs komunikacyjny dla serwisów:

```
1 int32 request_value
2 ---
3 bool success
4 string message
```

Typy pól odpowiadają typom podstawowym albo zdefiniowanym w paczkach jako `.msg`. Różnica pomiędzy interfejsem dla wiadomości a dla serwisów polega na pojawieniu się `---`. Jest to separator oddzielający żądanie wysyłane przez klienta od odpowiedzi wysyłanej przez serwer.

3.6 Utworzenie wiadomości dla ROS Action (.action)

W katalogu `action` (`~/tsr_workspaces/example_tsr_ws/src/example_tsr_msgs/action`) utworzyć plik tekstowy `GoToPose.action`, w którym zdefiniowany zostanie interfejs komunikacyjny dla wiadomości ROS Action. Wszystkie pliki w tym katalogu będą miały rozszerzenie `.action`. Do pliku należy wkleić przykład struktury wiadomości:

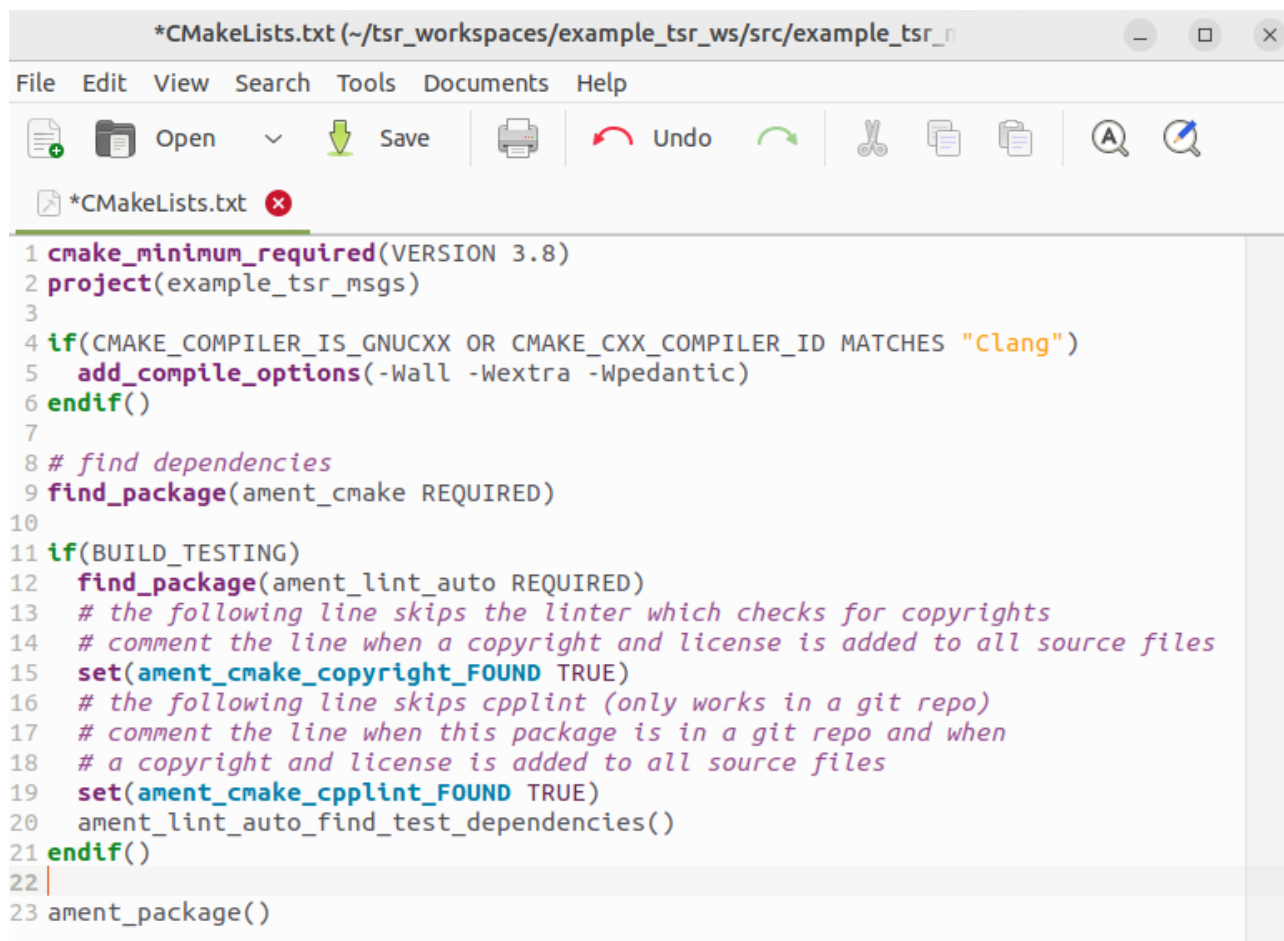
```

1 turtlesim/Pose goal_pose # Współrzędne docelowe
2 ---
3 bool success # Czy pomyślnie zakończono dojazd do celu
4 string message # Komunikat o wyniku
5 ---
6 float32 distance_remaining # Odległość do celu

```

3.7 Dodanie interfejsu do CMakeLists.txt.

W głównym katalogu pakietu `example_tsr_ws_msgs` znajduje się plik `CMakeLists.txt`, który należy otworzyć w celu edycji. Plik przed edycją wygląda następująco (Rys. 3.2):



Rys. 3.2: Plik `CMakeLists.txt` przed edycją dla paczki `example_tsr_msgs`

Do pliku należy dodać następujące linie:

```

1 find_package(action_msgs REQUIRED)
2 find_package(geometry_msgs REQUIRED)
3 find_package(turtlesim REQUIRED)
4 find_package(rosidl_default_generators REQUIRED)
5
6 rosidl_generate_interfaces(${PROJECT_NAME}
7   "msg/ExampleMsgType.msg"
8   "srv/ExampleSrvType.srv"
9   "action/GoToPose.action"
10  DEPENDENCIES action_msgs geometry_msgs turtlesim
11  )
12
13 ament_export_dependencies(rosidl_default_runtime)
14

```

Linia 1 jest niezbędna do prawidłowego wygenerowania wiadomości dla akcji. Linia 2 odpowiada za znalezienie wymaganych zależności od innych pakietów. W przykładowej wiadomości `.msg` wykorzystana została

wiadomość typu `Point` z paczki `geometry_msgs` z tego powodu ta zależność pojawia się w `CMakeLists.txt`. Zależność od pakietu `turtlesim` występuje w celu wysyłanym w akcji, z tego powodu ten pakiet również został dopisany do zależności. Zależność pojawiająca się w wiadomości od innej paczki należy również dodać do `rosidl_generate_interfaces` za `DEPENDENCIES` (linia 10).

Do wygenerowania nowych interfejsów komunikacyjnych niezbędne jest zaimportowanie pakietu `rosidl_default_generators` (linia 4).

W części `rosidl_generate_interfaces` (linie 6-11) należy dodać utworzone interfejsy komunikacyjne oraz zależności z nimi związane. Należy również wyeksportować zależności od `message runtime` (linia 13)

Po edycji plik powinien wyglądać następująco (Rys. 3.3):

```
1 cmake_minimum_required(VERSION 3.8)
2 project(example_tsr_msgs)
3
4 if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5   add_compile_options(-Wall -Wextra -Wpedantic)
6 endif()
7
8 # find dependencies
9 find_package(action_msgs REQUIRED)
10 find_package(ament_cmake REQUIRED)
11 find_package(geometry_msgs REQUIRED)
12 find_package(turtlesim REQUIRED)
13 find_package(rosidl_default_generators REQUIRED)
14
15
16 rosidl_generate_interfaces(${PROJECT_NAME}
17   "msg/ExampleMsgType.msg"
18   "srv/ExampleSrvType.srv"
19   "action/GoToPose.action"
20   DEPENDENCIES action_msgs geometry_msgs turtlesim
21 )
22
23 if(BUILD_TESTING)
24   find_package(ament_lint_auto REQUIRED)
25   # the following line skips the linter which checks for copyrights
26   # comment the line when a copyright and license is added to all source files
27   set(ament_cmake_copyright_FOUND TRUE)
28   # the following line skips cpplint (only works in a git repo)
29   # comment the line when this package is in a git repo and when
30   # a copyright and license is added to all source files
31   set(ament_cmake_cpplint_FOUND TRUE)
32   ament_lint_auto_find_test_dependencies()
33 endif()
34
35 ament_export_dependencies(rosidl_default_runtime)
36 ament_package()
--
```

Rys. 3.3: Plik `CMakeLists.txt` po edycji dla paczki `example_tsr_msgs`

3.8 Modyfikacja `package.xml`

Następnie należy dokonać edycji pliku `package.xml` i dodać następujące zależności, jeśli nie są dodane.

```
1 <depend>geometry_msgs</depend>
2 <depend>turtlesim</depend>
3 <depend>action_msgs</depend>
```

```

4 <buildtool_depend>roscpp</buildtool_depend>
5 <exec_depend>roscpp</exec_depend>
6 <member_of_group>roscpp</member_of_group>

```

Linie 1 należy dodać, gdyż korzystamy z wiadomości typu `Point` z pakietu `geometry_msgs` w tym pakiecie (`example_tsr_msgs`). Tak samo należy dodać zależności od innych pakietów (2-3) Linia 4 określa narzędzie budowania potrzebne do generowania kodu dla interfejsów komunikacyjnych (`msg`, `srv`, `action`). Linia 5 jest niezbędna do późniejszego wykorzystania interfejsów. `roscpp` jest nazwą grupy z zależnościami w utworzonym pakiecie.

Po edycji plik powinien wyglądać następująco (Rys. 3.4):

```

1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>example_tsr_msgs</name>
5   <version>0.0.0</version>
6   <description>TODO: Package description</description>
7   <maintainer email="ubuntu@todo.todo">ubuntu</maintainer>
8   <license>TODO: License declaration</license>
9
10  <buildtool_depend>ament_cmake</buildtool_depend>
11
12  <test_depend>ament_lint_auto</test_depend>
13  <test_depend>ament_lint_common</test_depend>
14
15  <depend>geometry_msgs</depend>
16  <depend>turtlesim</depend>
17  <depend>action_msgs</depend>
18
19  <buildtool_depend>roscpp</buildtool_depend>
20  <exec_depend>roscpp</exec_depend>
21  <member_of_group>roscpp</member_of_group>
22
23  <export>
24    <build_type>ament_cmake</build_type>
25  </export>
26 </package>

```

Rys. 3.4: Plik `package.xml` po edycji dla pakietu `example_tsr_msgs`

3.9 Budowanie pakietu

Należy przejść do głównego katalogu dla workspace'a (`example_tsr_ws`):

```

1 cd ~/tsr_workspaces/example_tsr_ws/src

```

następnie zbudować pojedynczy pakiet (można też zbudować cały workspace `colcon build`):

```

1 colcon build --packages-select example_tsr_msgs

```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

```

1 source install/setup.bash

```

3.10 Weryfikacja działania

Dla utworzonego interfejsu dla ROS Topic, ROS Service i ROS Action należy sprawdzić czy nowo utworzone interfejsy są widoczne.

```

1 ros2 interface show example_tsr_msgs/msg/ExampleMsgType
2 ros2 interface show example_tsr_msgs/srv/ExampleSrvType
3 ros2 interface show example_tsr_msgs/action/GoToPose

```

Prawidłowy rezultat obu zapytań przedstawia (Rys. 3.5):

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface show example_tsr_msgs/msg/ExampleMsgType
int32 id
string name
float32 value
bool is_valid
int32[] dynamic_integer_array
geometry_msgs/Point example_goal
    float64 x
    float64 y
    float64 z
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface show example_tsr_msgs/srv/ExampleSrvType
int32 request_value
---
bool success
string message
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 interface show example_tsr_msgs/action/GoToPose
turtlesim/Pose goal_pose # Współrzędne docelowe
    float32 x
    float32 y
    float32 theta
    float32 linear_velocity
    float32 angular_velocity
---
bool success # Czy pomyślnie zakończono dojazd do celu
string message # Komunikat o wyniku
---
float32 distance_remaining # Odległość do celu
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ █
```

Rys. 3.5: Wynik operacji sprawdzenia w terminalu struktury utworzonych interfejsów dla pakietu `example_tsr_msgs`.

Po użyciu polecenia sprawdzającego strukturę interfejsu nie mogą pojawić się błędy. Struktura ma być zgodna z tą jaką jest w dodanych plikach (katalogi `msg`, `srv`, `action`). Jeśli wyświetla się coś innego tzn., że należy ponownie zweryfikować poprawność konfiguracji i przejść procedurę jeszcze raz zwracając dokładniejszą uwagę na wykonywane kroki.

4 Procedura tworzenia nowego pakietu ROS2 dla wykonywalnych plików projektowych

Utworzony w tym kroku pakiet będzie zawierał niezbędne skrypty do uruchomienia projektu. Wszystkie utworzone węzły, pliki konfiguracyjne oraz inne dodatkowe pliki powinny znajdować się w tym pakiecie w odpowiednich katalogach:

- `config` - pliki konfiguracyjne np. `yaml`
- `launch` - pliki uruchamiające wiele węzłów/nodów
- `example_tsr_project` - jest to katalog nazywający się tak samo jak utworzony pakiet. W nim powinny znajdować się wszystkie pliki wykonywalne dla Python (rozszerzenie `.py`). Jeśli pakiet nazywa się inaczej to nazwa folderu, w którym powinny znajdować się pliki będzie inna np. utworzony jest pakiet `nowy_projekt2` to pliki `.py` powinny znajdować się w podkatalogu w tym pakiecie o nazwie `nowy_projekt2`.

4.1 Przejście do katalogu `src` w workspace

Należy w terminalu przejść do workspace'a (`example_tsr_ws`) do katalogu:

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

4.2 Utworzenie nowego pakietu

Uruchomić polecenie odpowiedzialne za utworzenie nowego pakietu o nazwie `example_tsr_ws_project`, w której będą umieszczane nowe interfejsy komunikacyjne.

```
1 ros2 pkg create --build-type ament_python example_tsr_project
```

4.3 Utworzenie podkatalogów

Pliki konfiguracyjne z parametrami znajdują się w pakiecie w katalogu `config`. Natomiast pliki `launch` pozwalają zarządzać uruchomieniem pakietu m.in. poprzez wczytanie odpowiednich parametrów konfiguracyjnych oraz uruchomienie nodów w odpowiedniej kolejności. Szczegóły dotyczące tych plików będą omówione w kolejnych tematach zajęć.

Należy przejść do katalogu z utworzonym pakietem (zamiast poleceń można skorzystać z okien):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

a następnie utworzyć katalog `launch` i `config`:

```
1 mkdir launch config
```

4.4 Modyfikacja `package.xml` w pakiecie `example_tsr_project`.

Należy dodać następujące zależności do pliku `package.xml`:

```
1 <exec_depend>rclpy</exec_depend>
2 <exec_depend>example_tsr_msgs</exec_depend>
3 <exec_depend>ros2launch</exec_depend>
```

Linia 1 odpowiada za dodanie zależności od biblioteki Python'a do ROS2. W linii 2 znajduje się dodanie zależności do utworzonego wcześniej pakietu z wiadomościami dla projektu `example_tsr_msgs`. Z kolei 3 linia odpowiada za dodanie zależności umożliwiających obsługę plików `launch`.

Po edycji plik `package.xml` powinien wyglądać następująco (Rys. 4.1):

```

1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://
  www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>example_tsr_project</name>
5   <version>0.0.0</version>
6   <description>TODO: Package description</description>
7   <maintainer email="ubuntu@todo.todo">ubuntu</maintainer>
8   <license>TODO: License declaration</license>
9
10  <test_depend>ament_copyright</test_depend>
11  <test_depend>ament_flake8</test_depend>
12  <test_depend>ament_pep257</test_depend>
13  <test_depend>python3-pytest</test_depend>
14
15  <exec_depend>rclpy</exec_depend>
16  <exec_depend>example_tsr_msgs</exec_depend>
17  <exec_depend>ros2launch</exec_depend>
18
19  <export>
20    <build_type>ament_python</build_type>
21  </export>
22 </package>

```

Rys. 4.1: Plik package.xml po edycji dla paczki example_tsr_project

4.5 Modyfikacja setup.py w pakiecie example_tsr_project.

Należy dodać następujące linie do pliku setup.py:

```

1 import os
2 from glob import glob
3
4 # W sekcji data_files dodać:
5 (os.path.join('share', package_name, 'launch'), glob('launch/*.py')),
6 (os.path.join('share', package_name, 'config'), glob('config/*.yaml')),
7 (os.path.join('share', package_name, 'rviz'), glob(os.path.join('config', '*.rviz'))),

```

Importy należy dodać na początku pliku a pozostałe linie w sekcji data_file. Po edycji plik setup.py powinien wyglądać następująco (Rys. ??fig:package_project_setup_py)):

```

1 from setuptools import find_packages, setup
2 import os
3 from glob import glob
4
5 package_name = 'example_tsr_project'
6
7 setup(
8     name=package_name,
9     version='0.0.0',
10    packages=find_packages(exclude=['test']),
11    data_files=[
12        ('share/ament_index/resource_index/packages',
13         ['resource/' + package_name]),
14        ('share/' + package_name, ['package.xml']),
15        (os.path.join('share', package_name, 'launch'), glob('launch/*.py')),
16        (os.path.join('share', package_name, 'config'), glob('config/*.yaml')),
17        (os.path.join('share', package_name, 'rviz'), glob(os.path.join('config', '*.rviz'))),
18    ],
19    install_requires=['setuptools'],
20    zip_safe=True,
21    maintainer='ubuntu',
22    maintainer_email='ubuntu@todo.todo'.

```

Rys. 4.2: Plik setup.py po edycji dla pakietu example_tsr_project

4.6 Budowanie paczki

Należy przejść do głównego katalogu dla workspace'a (`example_tsr_ws`):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczą paczkę (można też zbudować cały workspace):

```
1 colcon build --symlink-install --packages-select example_tsr_project
```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

```
1 source install/setup.bash
```

4.7 Weryfikacja działania

W terminalu należy wpisać:

```
1 ros2 pkg prefix example_tsr_project
```

W wyniku działania polecenia powinna pojawić się ścieżka z lokalizacją do miejsca instalacji pakietu (Rys. 4.3):

```
colcon build [1/1 done] [0 ongoing]
File Edit View Search Terminal Help
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ source install/setup.bash
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ros2 pkg prefix example_tsr_project
/home/ubuntu/tsr_workspaces/example_tsr_ws/install/example_tsr_project
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ █
```

Rys. 4.3: Wynik operacji sprawdzenia poprawności zbudowania pakietu.

5 Uruchomienie symulacji turtlesim

Do uruchomienia symulacji `turtlesim` należy w nowym terminalu wpisać:

```
1 ros2 run turtlesim turtlesim_node
```

Uruchomienie ręcznego sterowania z klawiatury:

```
1 ros2 run turtlesim turtle_teleop_key
```

5.1 Najważniejsze tematy

Dla pojedynczego utworzonego robota w przestrzeni nazw na przykładzie `turtle1` dostępne są następujące tematy dla ROS2 Topic:

- `/turtle1/cmd_vel` - prędkości sterujące robotem
- `/turtle1/color_sensor` - kolor
- `/turtle1/pose` - położenie robota

5.2 Najważniejsze serwisy

W podstawowym zakresie dostępne są następujące serwisy bez względu na robota:

- `/clear` - wyczyszczenie narysowanych ścieżek
- `/kill` - usunięcie robota
- `/reset` - reset środowiska do stanu początkowego
- `/spawn` - utworzenie nowego robota

Dla pojedynczego utworzonego robota w przestrzeni nazw na przykładzie `turtle1` dostępne są następujące serwisy:

- `/turtle1/set_pen` - ustawienie koloru pędzla do rysowania
- `/turtle1/teleport_absolute` - natychmiastowe przeniesienie robota do wskazanej lokalizacji
- `/turtle1/teleport_relative` - przeniesienie robota, współrzędne podawane w układzie robota

6 Dokumentacja i inne linki

Rozdział może zawierać dokumentację poleceń, linki do źródeł na podstawie, których powstała instrukcja oraz inne linki pozwalające poszerzyć wiedzę.

- Dokumentacja ROS Humble
<https://docs.ros.org/en/humble/index.html>
- Więcej o workspace
<https://docs.ros.org/en/dashing/Tutorials/Workspace/Creating-A-Workspace.html>
- Tworzenie nowej paczki C++/Python
<https://docs.ros.org/en/dashing/Tutorials/Creating-Your-First-ROS2-Package.html>
- Więcej o turtlesim
<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html>